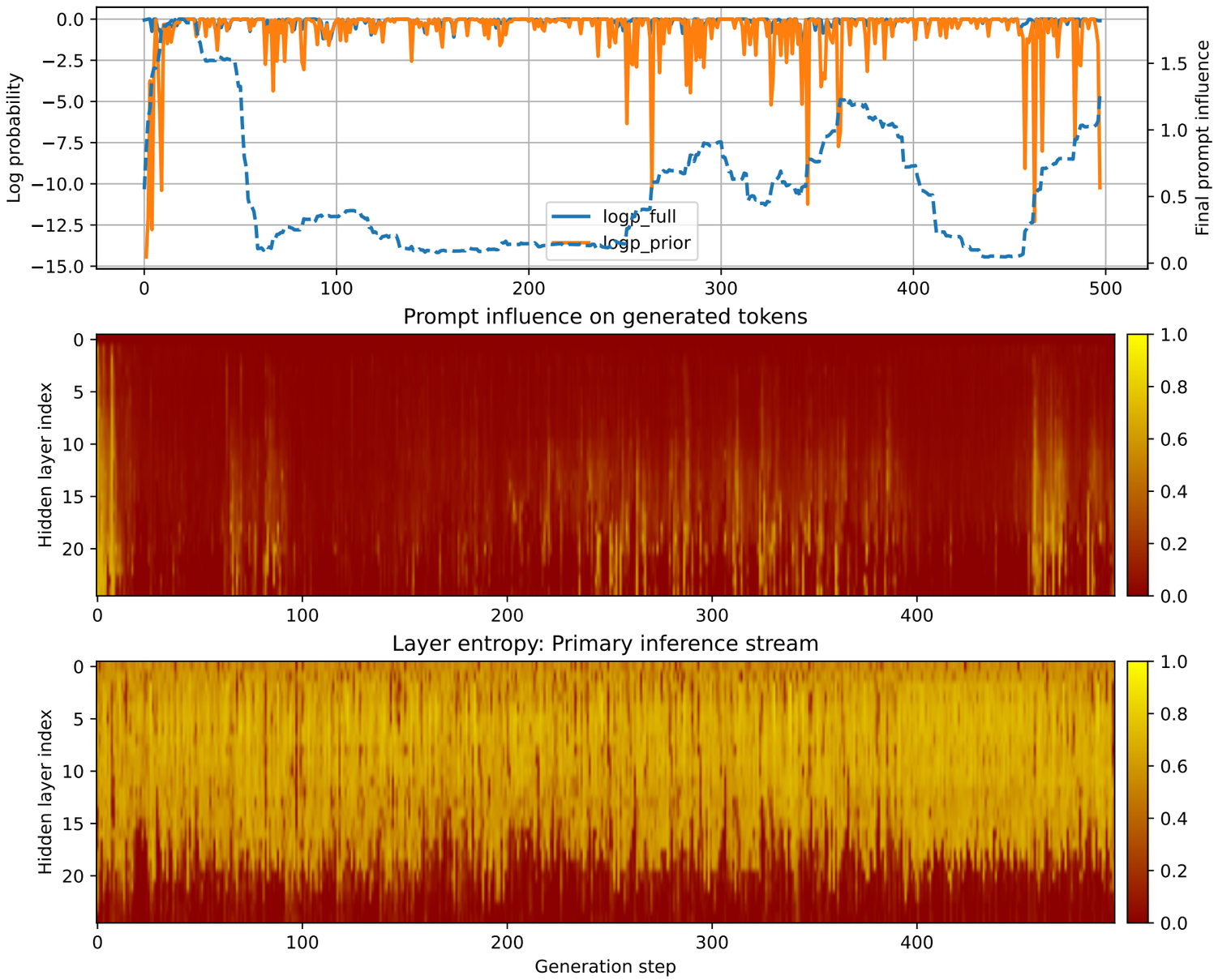
Prompt 1




```
PROMPT:
Task ID: PYC079  Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests.  Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC079  Your code must contain this required symbol:
def dijkstra  Implement this task: define
function dijkstra(graph,start) with nonnegative weights..  Start
your answer with the check-token line, then
provide the Python implementation.

MODEL RESPONSE:
# CHECK_TOKEN: dijkstra  # Definition for a weighted graph. #
class Graph: #     def __init__(self, vertices):
#         self.graph = {} #         self.weights = {} #
for vertex in vertices: #
self.graph[vertex] = {} #             self.weights[vertex] = 0  #
Function to calculate the shortest path in a
weighted graph using Dijkstra's algorithm. def dijkstra(graph,
start):     # Initialize the distance
dictionary and visited set     distance = {start: float('inf')}
  visited = set()      # Set the distance of
the starting vertex to 0     distance[start] = 0      # Create a
priority queue to store the vertices to be
visited     pq = [(0, start)]      # While the priority queue is
not empty:     while pq:         # Get the
current vertex with the smallest distance from the start
current_distance, current_vertex =
heapq.heappop(pq)          # If the current vertex has already
been visited, skip it          if current_vertex
in visited:             continue          # Mark the current
vertex as visited
visited.add(current_vertex)          # Update the distance of the
current vertex         for neighbor, weight
in graph[current_vertex].items():             distance[neighbor]
= min(distance[current_vertex],
distance[neighbor] + weight)          # Add the neighbors to the
priority queue with their updated distances
for neighbor, weight in graph[current_vertex].items():
 heapq.heappush(pq,
(distance[current_vertex] + weight, neighbor))      # The
shortest path from the start vertex to all other
vertices in the graph is the key of the distance dictionary
shortest_path = distance[start]     for
neighbor, weight in graph[start].items():         shortest_path =
min(shortest_path, distance[start] + weight)
return shortest_path  # Example usage: graph = {     'A': {'B':
2, 'C': 1},     'B': {'A': 1, 'D': 3},
'C': {'A': 2, 'D': 1},     'D': {'B': 3, 'C': 2} }  start = 'A'
shortest_path = dijkstra(graph, start)
print(f"The shortest path from {start} to all other vertices in
the graph is {shortest_path}.")
```